kj

kj

June 16, 2025

Listing 1: ./include/Stochastic/visualization.h

```
//
// Created by Nicolai Bergulff on 12/06/2025.
//
#ifndef STOCHASTIC_VISUALIZATION_H
#define STOCHASTIC_VISUALIZATION_H
#include <string>
#include <map>
#include <vector>

namespace Stochastic {
    class Vessel;
    class Reaction;

    class Visualization {
    public:
        //R2
        static std::string generateDotGraph(const Vessel& vessel);
        //R2
        static std::string generateTextDescription(const Vessel& vessel);

        //R6
        static void saveTrajectoryToCSV(const std::string& filename, const std::map<double,
↪std::map<std::string, size_t>>& trajectory);
        static void saveTrajectoryToGnuplot(const std::string& filename, const std::map<double,
↪std::map<std::string, size_t>>& trajectory);
    };
}
#endif // STOCHASTIC_VISUALIZATION_H
```

Listing 2: ./include/Stochastic/observer.h

```
//
// Created by Nicolai Bergulff on 12/06/2025.
//
#ifndef STOCHASTIC_OBSERVER_H
#define STOCHASTIC_OBSERVER_H
#include <map>
#include <vector>
#include <string>
#include <functional>

namespace Stochastic {
    class Species;
    // R7
    class Observer {
    public:
        virtual ~Observer() = default;
        virtual void observe(double time, const std::vector<Species*>& species) = 0;
    };
```

```
19
20      class TrajectoryObserver : public Observer {
21          std::map<double, std::map<std::string, size_t>> trajectory_;
22      public:
23          void observe(double time, const std::vector<Species*>& species) override;
24          const std::map<double, std::map<std::string, size_t>>& getTrajectory() const;
25          void saveToFile(const std::string& filename) const;
26      };
27
28      template<typename T>
29      class FunctionObserver : public Observer {
30          std::function<T(double, const std::vector<Species*>&)> function_;
31          std::vector<T> values_;
32      public:
33          // R7
34          explicit FunctionObserver(std::function<T(double, const std::vector<Species*>&)> func);
35          void observe(double time, const std::vector<Species*>& species) override;
36          const std::vector<T>& getValues() const;
37      };
38
39      class PeakObserver : public Observer {
40          std::string species_name_;
41          double peak_time_ = 0.0;
42          size_t peak_value_ = 0;
43      public:
44          explicit PeakObserver(const std::string& speciesName);
45          void observe(double time, const std::vector<Species*>& species) override;
46          std::pair<double, size_t> getPeak() const;
47      };
48  }
49  #endif // STOCHASTIC_OBSERVER_H
```

Listing 3: ./include/Stochastic/stochastic.h

```
1   //
2   // Created by Nicolai Bergulff on 14/06/2025.
3   //
4
5   #ifndef STOCHASTIC_H
6   #define STOCHASTIC_H
7
8   #pragma once
9
10  // Main header that includes all components of the stochastic simulation library
11  #include "species.h"
12  #include "reaction.h"
13  #include "vessel.h"
14  #include "simulator.h"
15  #include "observer.h"
16  #include "visualization.h"
17  #include "symbol_table.h"
18  #include "exceptions.h"
19
20  namespace Stochastic {
21      Vessel seir(uint32_t N);  // Covid-19 SEIHR model
22      Vessel circadian_rhythm();
23
24  }
25
26
27  // This header provides the complete stochastic simulation library interface
28  // Include this single header to access all functionality
```

```

#endif //STOCHASTIC_H
```

Listing 4: ./include/Stochastic/exceptions.h

```
//
// Created by Nicolai Bergulff on 12/06/2025.
//
#ifndef STOCHASTIC_EXCEPTIONS_H
#define STOCHASTIC_EXCEPTIONS_H
#include <stdexcept>
#include <string>

namespace Stochastic {
    class SymbolTableException : public std::runtime_error {
    public:
        explicit SymbolTableException(const std::string& msg) : std::runtime_error(msg) {}
    };

    class SimulationException : public std::runtime_error {
    public:
        explicit SimulationException(const std::string& msg) : std::runtime_error(msg) {}
    };
}
#endif // STOCHASTIC_EXCEPTIONS_H
```

Listing 5: ./include/Stochastic/simulator.h

```
//
// Created by Nicolai Bergulff on 12/06/2025.
//
#ifndef STOCHASTIC_SIMULATOR_H
#define STOCHASTIC_SIMULATOR_H
#include <thread>
#include <future>
#include <vector>
#include <functional>
#include "vessel.h"
#include "observer.h"

namespace Stochastic {
    class Simulator {
    public:
        // R4
        static void runSimulation(Vessel& vessel, double endTime, Observer* observer = nullptr);

        //R8
        template<typename ResultType>
        static std::vector<ResultType> runParallelSimulations(
            Vessel& vessel,
            double endTime,
            size_t numSimulations,
            std::function<ResultType(const Vessel&)> resultExtractor,
            size_t numThreads = std::thread::hardware_concurrency()
        );
    };
}
#endif // STOCHASTIC_SIMULATOR_H


/*#ifndef SIMULATOR_H
#define SIMULATOR_H
```

```
#pragma once
#include <vector>
#include <thread>
#include <future>
#include "vessel.h"
#include "observer.h"

namespace Stochastic {

    class Simulator {
    public:
        // Run a single simulation
        static void runSimulation(Vessel& vessel, double endTime, Observer* observer = nullptr);

        // Run multiple simulations in parallel
        template<typename ResultType>
        static std::vector<ResultType> runParallelSimulations(
            Vessel& vessel,
            double endTime,
            size_t numSimulations,
            std::function<ResultType(const Vessel&)> resultExtractor,
            size_t numThreads = std::thread::hardware_concurrency()
        ) {
            std::vector<ResultType> results(numSimulations);
            std::vector<std::future<void>> futures;

            // Divide simulations among threads
            size_t simulationsPerThread = numSimulations / numThreads;
            size_t remainingSimulations = numSimulations % numThreads;

            size_t startIndex = 0;

            for (size_t i = 0; i < numThreads; ++i) {
                size_t count = simulationsPerThread + (i < remainingSimulations ? 1 : 0);
                size_t endIndex = startIndex + count;

                if (count == 0) continue;

                futures.push_back(std::async(std::launch::async, [&, startIndex, endIndex]() {
                    for (size_t j = startIndex; j < endIndex; ++j) {
                        // Create a copy of the vessel for this simulation
                        Vessel vesselCopy = vessel;
                        runSimulation(vesselCopy, endTime);
                        results[j] = resultExtractor(vesselCopy);
                    }
                }));

                startIndex = endIndex;
            }

            // Wait for all threads to complete
            for (auto& future : futures) {
                future.wait();
            }

            return results;
        }
    };

} // namespace stochastic
#endif //SIMULATOR_H*/
```

Listing 6: ./include/Stochastic/reaction.h

```
//
// Created by Nicolai Bergulff on 12/06/2025.
//
#ifndef STOCHASTIC_REACTION_H
#define STOCHASTIC_REACTION_H

#include <vector>
#include <memory>

namespace Stochastic {
    class Species;

    class ReactionBuilder {
        std::vector<std::pair<size_t, size_t>> reactant_ids_;
        std::vector<std::pair<size_t, size_t>> product_ids_;
        double rate_;
        bool has_rate_;
    public:
        ReactionBuilder();
        ReactionBuilder(size_t species_id);
        ReactionBuilder(const std::vector<size_t>& reactants);

        ReactionBuilder operator+(const Species& species) const;
        ReactionBuilder operator>>(double rate) const;
        ReactionBuilder operator>>=(const Species& product) const;
        ReactionBuilder operator>>=(const ReactionBuilder& products) const;

        const std::vector<std::pair<size_t, size_t>>& reactants() const;
        const std::vector<std::pair<size_t, size_t>>& products() const;
        double rate() const;
        bool has_rate() const;
    };

    class Reaction {
        std::vector<std::pair<size_t, size_t>> reactant_ids_;
        std::vector<std::pair<size_t, size_t>> product_ids_;
        double rate_;
        double current_delay_;
        size_t id_;
    public:
        Reaction(const std::vector<std::pair<size_t, size_t>>& reactants,
                 const std::vector<std::pair<size_t, size_t>>& products,
                 double rate, size_t id);

        const std::vector<std::pair<size_t, size_t>>& reactants() const;
        const std::vector<std::pair<size_t, size_t>>& products() const;
        double rate() const;
        double delay() const;
        size_t id() const;

        void set_delay(double delay);
        bool can_proceed(const std::vector<std::shared_ptr<Species>>& species) const;
        double calculate_propensity(const std::vector<std::shared_ptr<Species>>& species) const;
        void execute(std::vector<std::shared_ptr<Species>>& species) const;
    };

    struct ReactionComparator {
        bool operator()(const std::shared_ptr<Reaction>& a, const std::shared_ptr<Reaction>& b)
↪const;
    };
```

```
}

#endif // STOCHASTIC_REACTION_H
```

Listing 7: ./include/Stochastic/vessel.h

```
//
// Created by Nicolai Bergulff on 12/06/2025.
//
#ifndef STOCHASTIC_VESSEL_H
#define STOCHASTIC_VESSEL_H
#pragma once

#include <vector>
#include <string>
#include <memory>
#include <functional>
#include <random>
#include "species.h"
#include "reaction.h"
#include "symbol_table.h"

namespace Stochastic {
    // R4
    class Vessel {
        std::string name_;
        std::vector<std::shared_ptr<Species>> species_;
        std::vector<std::shared_ptr<Reaction>> reactions_;
        // R3
        SymbolTable<std::string, size_t> species_name_to_id_;
        size_t next_species_id_;
        size_t next_reaction_id_;
        size_t environment_id_;
        std::mt19937 rng_{};

    public:
        explicit Vessel(const std::string& name);

        // API to add species and reactions
        std::shared_ptr<Species> add(const std::string& name, size_t initial_count);
        void add(const ReactionBuilder& builder);

        // Access and query
        std::shared_ptr<Species> environment() const;
        std::shared_ptr<Species> get_species(const std::string& name) const;
        const std::vector<std::shared_ptr<Species>>& get_species() const;

        // R6
        void simulate(double end_time,
                      std::function<void(double, const std::vector<std::shared_ptr<Species>>&)>
↪observer = nullptr);


        std::string reaction_to_string(size_t idx) const;
        // R2
        std::string to_string() const;
        std::string to_dot() const;

    private:
        std::shared_ptr<Species> add_species(const std::string& name, size_t initial_count);
        void add_reaction(const std::vector<std::pair<size_t, size_t>>& reactants,
                  const std::vector<std::pair<size_t, size_t>>& products,
```

```
                    double rate);
        void update_all_delays();
        std::shared_ptr<Reaction> find_next_reaction();
    };

} // namespace Stochastic

#endif // STOCHASTIC_VESSEL_H
```

Listing 8: ./include/Stochastic/species.h

```
//
// Created by Nicolai Bergulff on 12/06/2025.
//
#ifndef STOCHASTIC_SPECIES_H
#define STOCHASTIC_SPECIES_H
#include <string>

namespace Stochastic {
    class ReactionBuilder;

    class Species {
        std::string name_;
        size_t count_;
        size_t id_;
    public:
        Species(const std::string& name, size_t initial_count, size_t id);

        const std::string& name() const;
        size_t count() const;
        size_t id() const;

        void set_count(size_t count);
        void increment_count(size_t amount = 1);
        void decrement_count(size_t amount = 1);

        // R1
        ReactionBuilder operator+(const Species& other) const;
        ReactionBuilder operator>>(double rate) const;
        ReactionBuilder operator>>=(const Species& product) const;
    };
}
#endif // STOCHASTIC_SPECIES_H
```

Listing 9: ./include/Stochastic/symbol_table.h

```
//
// Created by Nicolai Bergulff on 12/06/2025.
//
#ifndef STOCHASTIC_SYMBOL_TABLE_H
#define STOCHASTIC_SYMBOL_TABLE_H

#include <unordered_map>
#include <vector>
#include "exceptions.h"

namespace Stochastic {
    // R3
    template<typename Key, typename Value>
    class SymbolTable {
        std::unordered_map<Key, Value> table_;
    public:
```

```
        void add(const Key& key, const Value& value) {
            if (table_.find(key) != table_.end())
                throw SymbolTableException("Symbol already exists: " + key);
            table_[key] = value;
        }

        const Value& get(const Key& key) const {
            auto it = table_.find(key);
            if (it == table_.end())
                throw SymbolTableException("Symbol not found: " + key);
            return it->second;
        }

        bool contains(const Key& key) const {
            return table_.find(key) != table_.end();
        }

        void update(const Key& key, const Value& value) {
            if (!contains(key))
                throw SymbolTableException("Cannot update non-existent symbol: " + key);
            table_[key] = value;
        }

        std::vector<Key> keys() const {
            std::vector<Key> result;
            for (const auto& kv : table_)
                result.push_back(kv.first);
            return result;
        }
    };
}
#endif // STOCHASTIC_SYMBOL_TABLE_H
```

Listing 10: ./examples/seihr_main.cpp

```
//
// Created by Nicolai Bergulff on 14/06/2025.
//
// examples/seir_main.cpp
#include "../include/Stochastic/stochastic.h"
#include <iostream>

using namespace Stochastic;

int main() {
    auto v = seir(10000);  // Uses function defined in covid_seihr.cpp

    TrajectoryObserver obs;
    Simulator::runSimulation(v, 100.0, &obs);

    obs.saveToFile("seihr_trajectory.csv");

    std::cout << "SEIHR simulation complete.\n";
    return 0;
}
```

Listing 11: ./examples/covid_seihr.cpp

```
//
// Created by Nicolai Bergulff on 14/06/2025.
//
#include "../include/Stochastic/stochastic.h"
```

```
namespace Stochastic {

    Vessel seir(uint32_t N) {
        auto v = Vessel("COVID19 SEIHR: " + std::to_string(N));

        const auto eps = 0.0009;
        const auto I0 = size_t(std::round(eps * N));
        const auto E0 = size_t(std::round(eps * N * 15));
        const auto S0 = N - I0 - E0;

        const auto R0 = 2.4;
        const auto alpha = 1.0 / 5.1;
        const auto gamma = 1.0 / 3.1;
        const auto beta = R0 * gamma;
        const auto P_H = 9e-3;
        const auto kappa = gamma * P_H * (1.0 - P_H);
        const auto tau = 1.0 / 10.12;

        const auto S = v.add("S", S0);
        const auto E = v.add("E", E0);
        const auto I = v.add("I", I0);
        const auto H = v.add("H", 0);
        const auto R = v.add("R", 0);

        v.add((*S + *I) >> beta / N >>= *E + *I);
        v.add(*E >> alpha >>= *I);
        v.add(*I >> gamma >>= *R);
        v.add(*I >> kappa >>= *H);
        v.add(*H >> tau >>= *R);

        return v;
    }

}
```

Listing 12: ./examples/circadian_rythm.cpp

```
//
// Created by Nicolai Bergulff on 14/06/2025.
//
#include "../include/Stochastic/stochastic.h"

namespace Stochastic {

    Vessel circadian_rhythm() {
        const auto alphaA = 50;
        const auto alphaMA = 500;
        const auto alphaR = 0.01;
        const auto alphaR_R = 50;
        const auto betaA = 50;
        const auto betaR = 5;
        const auto gammaA = 1;
        const auto gammaR = 1;
        const auto gammaC = 2;
        const auto deltaA = 1;
        const auto deltaR = 0.2;
        const auto deltaMA = 10;
        const auto deltaMR = 0.5;
        const auto thetaA = 50;
        const auto thetaR = 100;

        auto v = Vessel("Circadian Rhythm");
```

```
26
27          const auto env = v.environment();
28          const auto DA = v.add("DA", 10);   // Increase to allow more reactions
29          const auto D_A = v.add("D_A", 0);
30          const auto DR = v.add("DR", 10);   // Same here
31          const auto D_R = v.add("D_R", 0);
32          const auto MA = v.add("MA", 5);    // Start with some mRNA
33          const auto MR = v.add("MR", 5);
34          const auto A  = v.add("A", 5);     //     Needed for initial reactions
35          const auto R  = v.add("R", 5);     //     Needed for complexation
36          const auto C  = v.add("C", 0);
37
38
39          v.add((*A + *DA) >> gammaA >>= *D_A);
40          v.add(*D_A >> thetaA >>= *DA + *A);
41          v.add((*A + *DR) >> gammaR >>= *D_R);
42          v.add(*D_R >> thetaR >>= *DR + *A);
43
44          v.add(*D_A >> alphaA >>= *MA + *D_A);
45          v.add(*DA >> alphaMA >>= *MA + *DA);
46          v.add(*D_R >> alphaR >>= *MR + *D_R);
47          v.add(*DR >> alphaR_R >>= *MR + *DR);
48
49          v.add(*MA >> betaA >>= *MA + *A);
50          v.add(*MR >> betaR >>= *MR + *R);
51
52          v.add((*A + *R) >> gammaC >>= *C);
53          v.add(*C >> deltaA >>= *R);
54
55          v.add(*A >> deltaA >>= *env);
56          v.add(*R >> deltaR >>= *env);
57          v.add(*MA >> deltaMA >>= *env);
58          v.add(*MR >> deltaMR >>= *env);
59
60          return v;
61      }
62
63  }
```

Listing 13: ./examples/circadian_main.cpp

```
1  //
2  // Created by Nicolai Bergulff on 14/06/2025.
3  //
4  #include "../include/Stochastic/stochastic.h"
5  #include <iostream>
6
7  using namespace Stochastic;
8
9  int main() {
10     auto v = circadian_rhythm();
11
12     TrajectoryObserver obs;
13     Simulator::runSimulation(v, 48.0, &obs);
14     obs.saveToFile("circadian_trajectory.csv");
15
16     std::cout << "Circadian simulation complete.\n";
17     return 0;
18 }
```

Listing 14: ./examples/main.cpp

```cpp
#include "../include/Stochastic/stochastic.h"
#include <filesystem>
#include <iostream>
#include <numeric>

using namespace Stochastic;

int main() {
    std::filesystem::create_directory("output1");

    // === Circadian Rhythm (R5, R6, R7) ===
    Vessel circadian = circadian_rhythm();
    TrajectoryObserver traj;
    Simulator::runSimulation(circadian, 200.0, &traj);

    const auto& trajectory = traj.getTrajectory();
    std::cout << "[DEBUG] Time points recorded: " << trajectory.size() << "\n";

    if (trajectory.empty()) {
        std::cerr << "[ERROR] No data recorded! Check initial species counts "
                     "(e.g., A, DA, DR must start > 0).\n";
    } else {
        // Print the first 10 time points
        size_t counter = 0;
        for (auto it = trajectory.begin(); it != trajectory.end() && counter < 10; ++it,
↪++counter) {
            const auto& [t, snapshot] = *it;
            std::cout << t << ": ";
            for (const auto& [name, count] : snapshot)
                std::cout << name << "=" << count << " ";
            std::cout << "\n";
        }

        // Save to CSV
        Visualization::saveTrajectoryToCSV("output1/circadian.csv", trajectory);
        std::cout << "[Circadian] Saved output to output1/circadian.csv\n";
    }

    // === SEIHR Example ===
    Vessel covid = seir(10000);
    PeakObserver peak("H");
    Simulator::runSimulation(covid, 100.0, &peak);
    auto [time, value] = peak.getPeak();
    std::cout << "[Covid] Peak hospitalized: " << value << " at time " << time << "\n";

    auto peaks = Simulator::runParallelSimulations<size_t>(
        covid, 100.0, 100,
        [](const Vessel& v) {
            PeakObserver local("H");
            Simulator::runSimulation(const_cast<Vessel&>(v), 100.0, &local);
            return local.getPeak().second;
        });

    double avg_peak = std::accumulate(peaks.begin(), peaks.end(), 0.0) / peaks.size();
    std::cout << "[Covid] Average peak (100 sims): " << avg_peak << "\n";

    std::cout << "\nRun this to plot circadian:\n";
    std::cout << "gnuplot -e \"set datafile separator ','; set terminal png size 1000,600; "
                 "set output 'output1/circadian.png'; "
                 "plot 'output1/circadian.csv' using 1:2 with lines title columnhead(2), "
                 "'output1/circadian.csv' using 1:3 with lines title columnhead(3), "
```

```
                "'output1/circadian.csv' using 1:4 with lines title columnhead(4)\"\n";

    return 0;
}
```

Listing 15: ./benchmarks/benchmark.cpp

```
//
// Created by Nicolai Bergulff on 14/06/2025.
//
#include "../include/Stochastic/stochastic.h"
#include <iostream>
#include <chrono>
#include <fstream>
#include <thread>

//R10
using namespace Stochastic;
using Clock = std::chrono::high_resolution_clock;

int main() {
    const size_t N = 10000;
    const double end_time = 100.0;
    const size_t runs = 100;
    const size_t threads = std::thread::hardware_concurrency();

    std::cout << "Benchmarking SEIHR Covid-19 Model...\n";

    // Build model once for parallel simulation
    Vessel base_model = seir(N);

    // --- Single Core Benchmark ---
    auto t1 = Clock::now();
    for (size_t i = 0; i < runs; ++i) {
        Vessel copy = seir(N);
        copy.simulate(end_time);
    }
    auto t2 = Clock::now();
    std::chrono::duration<double> serial_duration = t2 - t1;

    std::cout << "Single-threaded (" << runs << " runs): "
              << serial_duration.count() << " seconds\n";

    // --- Multi Core Benchmark ---
    auto t3 = Clock::now();
    auto results = Simulator::runParallelSimulations<size_t>(
        base_model, end_time, runs,
        [](const Vessel& v) {
            auto* sp = v.get_species("H").get();
            return sp->count();
        },
        threads
    );
    auto t4 = Clock::now();
    std::chrono::duration<double> parallel_duration = t4 - t3;

    std::cout << "Multithreaded (" << threads << " threads, " << runs << " runs): "
              << parallel_duration.count() << " seconds\n";

    // --- Export benchmark results ---
    std::ofstream out("benchmark_results.csv");
    if (!out) {
```

```
        std::cerr << "Failed to write benchmark_results.csv\n";
        return 1;
    }

    out << "Mode,Time\n";
    out << "Single," << serial_duration.count() << "\n";
    out << "Parallel," << parallel_duration.count() << "\n";

    std::cout << "Benchmark results written to benchmark_results.csv\n";
    return 0;
}
```

Listing 16: ./src/visualization.cpp

```
//
// Created by Nicolai Bergulff on 12/06/2025.
//
#include "../include/Stochastic/stochastic.h"
#include <fstream>
#include <stdexcept>

namespace Stochastic {
    // R2
    std::string Visualization::generateDotGraph(const Vessel& vessel) {
        return vessel.to_dot();
    }
    // R2
    std::string Visualization::generateTextDescription(const Vessel& vessel) {
        return vessel.to_string();
    }
    // R6
    void Visualization::saveTrajectoryToCSV(const std::string& filename, const std::map<double,
↪std::map<std::string, size_t>>& trajectory) {
        std::ofstream file(filename);
        if (!file.is_open()) throw std::runtime_error("Failed to open file " + filename);

        if (trajectory.empty()) return;
        const auto& headers = trajectory.begin()->second;

        file << "time";
        for (const auto& [name, _] : headers) file << "," << name;
        file << "\n";

        for (const auto& [time, values] : trajectory) {
            file << time;
            for (const auto& [_, count] : values) file << "," << count;
            file << "\n";
        }
    }
    // R6
    void Visualization::saveTrajectoryToGnuplot(const std::string& filename, const
↪std::map<double, std::map<std::string, size_t>>& trajectory) {
        std::ofstream file(filename);
        if (!file.is_open()) throw std::runtime_error("Failed to open file " + filename);
        for (const auto& [time, values] : trajectory) {
            file << time;
            for (const auto& [_, count] : values) file << "\t" << count;
            file << "\n";
        }
    }

} // namespace Stochastic
```

Listing 17: ./src/vessel.cpp

```cpp
//
// Created by Nicolai Bergulff on 12/06/2025.
//
#include "../include/Stochastic/stochastic.h"
#include "../include/Stochastic/vessel.h"
#include <sstream>
#include <algorithm>
#include <iostream>
#include <random>

namespace Stochastic {
// R4
Vessel::Vessel(const std::string& name)
    : name_(name), next_species_id_(0), next_reaction_id_(0), rng_(std::random_device{}()) {
    environment_id_ = add_species("environment", 0)->id();

}
// R3
std::shared_ptr<Species> Vessel::add(const std::string& name, size_t initial_count) {
    return add_species(name, initial_count);
}

std::shared_ptr<Species> Vessel::environment() const {
    return species_[environment_id_];
}
// R4
void Vessel::add(const ReactionBuilder& builder) {
    if (!builder.has_rate())
        throw SimulationException("Reaction must have a rate specified");
    add_reaction(builder.reactants(), builder.products(), builder.rate());
}
// R3
std::shared_ptr<Species> Vessel::add_species(const std::string& name, size_t initial_count) {
    if (species_name_to_id_.contains(name))
        throw SymbolTableException("Species already exists: " + name);
    auto sp = std::make_shared<Species>(name, initial_count, next_species_id_);
    species_.push_back(sp);
    species_name_to_id_.add(name, next_species_id_);
    return species_[next_species_id_++];
}
// R4
void Vessel::add_reaction(const std::vector<std::pair<size_t, size_t>>& reactants,
                          const std::vector<std::pair<size_t, size_t>>& products,
                          double rate) {
    auto r = std::make_shared<Reaction>(reactants, products, rate, next_reaction_id_++);
    reactions_.push_back(r);
}

// R4
std::shared_ptr<Reaction> Vessel::find_next_reaction() {
    return *std::min_element(reactions_.begin(), reactions_.end(), ReactionComparator());
}
// R4
void Vessel::update_all_delays() {
    for (auto& r : reactions_) {
        double p = r->calculate_propensity(species_);
        r->set_delay(p > 0 ? std::exponential_distribution<>(p)(rng_) :
↪std::numeric_limits<double>::infinity());
```

```
    }
}
// R4 & R6
    void Vessel::simulate(double end_time, std::function<void(double, const
↪std::vector<std::shared_ptr<Species>>&)> observer) {
    double time = 0.0;
    while (time <= end_time) {
        update_all_delays();
        auto next = find_next_reaction();

        //      DEBUG
        std::cout << "[SIM] next reaction ID: " << next->id()
                  << ", delay: " << next->delay() << std::endl;

        double dt = next->delay();
        if (dt == std::numeric_limits<double>::infinity()) {
            std::cout << "[SIM] All propensities are zero     exiting.\n";
            break;
        }

        time += dt;
        std::cout << "[DEBUG] Time: " << time << ", Executing reaction ID " << next->id() <<
↪std::endl;
        next->execute(species_);

        //      DEBUG
        std::cout << "[SIM] Executed reaction " << next->id()
                  << " at time " << time << std::endl;

        if (observer) observer(time, species_);
    }
}

// R3
std::shared_ptr<Species> Vessel::get_species(const std::string& name) const {
    return species_[species_name_to_id_.get(name)];
}
const std::vector<std::shared_ptr<Species>>& Vessel::get_species() const {
    return species_;
}
// R2
std::string Vessel::reaction_to_string(size_t idx) const {
    std::stringstream ss;
    auto& r = reactions_[idx];
    for (const auto& [id, count] : r->reactants()) ss << species_[id]->name() << " + ";
    ss << "--(" << r->rate() << ")--> ";
    for (const auto& [id, count] : r->products()) ss << species_[id]->name() << " + ";
    return ss.str();
}
    // R2
std::string Vessel::to_string() const {
    std::stringstream ss;
    for (const auto& s : species_) ss << s->name() << "=" << s->count() << ", ";
    return ss.str();
}
    // R2
std::string Vessel::to_dot() const {
    std::stringstream ss;
    ss << "digraph {\n";
    for (const auto& s : species_) ss << "s" << s->id() << "[label=\"" << s->name() <<
↪"\",shape=box];\n";
```

```
    for (const auto& r : reactions_) {
        ss << "r" << r->id() << "[label=\"" << r->rate() << "\",shape=oval];\n";
        for (const auto& [id, count] : r->reactants()) ss << "s" << id << " -> r" << r->id() << ";\n";
        for (const auto& [id, count] : r->products()) ss << "r" << r->id() << " -> s" << id << ";\n";
    }
    ss << "}\n";
    return ss.str();
}

} // namespace Stochastic
```

Listing 18: ./src/species.cpp

```
//
// Created by Nicolai Bergulff on 12/06/2025.
//
#include "../include/Stochastic/stochastic.h"
#include <stdexcept>

namespace Stochastic {

    Species::Species(const std::string& name, size_t initial_count, size_t id)
        : name_(name), count_(initial_count), id_(id) {}

    const std::string& Species::name() const { return name_; }
    size_t Species::count() const { return count_; }
    size_t Species::id() const { return id_; }

    void Species::set_count(size_t count) { count_ = count; }
    void Species::increment_count(size_t amount) { count_ += amount; }
    // R4
    void Species::decrement_count(size_t amount) {
        if (count_ < amount)
            throw SimulationException("Cannot decrement species " + name_ + " below zero");
        count_ -= amount;
    }
    // R1
    ReactionBuilder Species::operator+(const Species& other) const {
        return ReactionBuilder({id_, other.id_});
    }
    ReactionBuilder Species::operator>>(double rate) const {
        ReactionBuilder builder(id_);
        return builder >> rate;
    }
    ReactionBuilder Species::operator>>=(const Species& product) const {
        ReactionBuilder builder(id_);
        return (builder >> 1.0) >>= product;
    }

} // namespace Stochastic
```

Listing 19: ./src/simulator.cpp

```
//
// Created by Nicolai Bergulff on 12/06/2025.
//
#include "../include/Stochastic/stochastic.h"
#include <future>
#include <thread>

```

```cpp
namespace Stochastic {
    // R4
    void Simulator::runSimulation(Vessel& vessel, double endTime, Observer* observer) {
        std::function<void(double, const std::vector<std::shared_ptr<Species>>&)> cb = nullptr;

        if (observer) {
            cb = [&](double t, const std::vector<std::shared_ptr<Species>>& sps) {
                std::vector<Species*> raw;
                for (const auto& sp : sps) raw.push_back(sp.get());
                observer->observe(t, raw);
            };
        }

        vessel.simulate(endTime, cb);
    }
    // R8
    template<typename ResultType>
    std::vector<ResultType> Simulator::runParallelSimulations(
        Vessel& vessel,
        double endTime,
        size_t numSimulations,
        std::function<ResultType(const Vessel&)> resultExtractor,
        size_t numThreads) {

        std::vector<ResultType> results(numSimulations);
        std::vector<std::future<void>> futures;
        size_t simsPerThread = numSimulations / numThreads;
        size_t leftover = numSimulations % numThreads;

        size_t index = 0;
        for (size_t t = 0; t < numThreads; ++t) {
            size_t count = simsPerThread + (t < leftover ? 1 : 0);
            size_t start = index, end = start + count;
            // R8
            futures.push_back(std::async(std::launch::async, [&, start, end]() {
                for (size_t i = start; i < end; ++i) {
                    Vessel copy = vessel;
                    copy.simulate(endTime);
                    results[i] = resultExtractor(copy);
                }
            }));
            index = end;
        }

        for (auto& f : futures) f.get();
        return results;
    }

    // R8
    template std::vector<size_t> Simulator::runParallelSimulations<size_t>(
        Vessel&, double, size_t, std::function<size_t(const Vessel&)>, size_t);

} // namespace Stochastic
```

Listing 20: ./src/observer.cpp

```cpp
//
// Created by Nicolai Bergulff on 14/06/2025.
//
#include "../include/Stochastic/stochastic.h"
#include <fstream>
#include <iostream>
```

```cpp
#include <stdexcept>

namespace Stochastic {
    // R6
    void TrajectoryObserver::observe(double time, const std::vector<Species*>& species) {
        std::cout << "[OBSERVER] Recording at time: " << time << std::endl;

        std::map<std::string, size_t> snapshot;
        for (const auto* s : species) snapshot[s->name()] = s->count();
        trajectory_[time] = std::move(snapshot);
    }

    const std::map<double, std::map<std::string, size_t>>& TrajectoryObserver::getTrajectory() const {
        return trajectory_;
    }
    // R6
    void TrajectoryObserver::saveToFile(const std::string& filename) const {
        std::ofstream out(filename);
        if (!out) throw std::runtime_error("Cannot open file: " + filename);

        if (trajectory_.empty()) return;

        const auto& first_row = trajectory_.begin()->second;
        out << "time";
        for (const auto& [name, _] : first_row) out << "," << name;
        out << "\n";

        for (const auto& [time, snapshot] : trajectory_) {
            out << time;
            for (const auto& [_, count] : snapshot) out << "," << count;
            out << "\n";
        }
    }
    // R7
    PeakObserver::PeakObserver(const std::string& species_name)
        : species_name_(species_name), peak_time_(0.0), peak_value_(0) {}

    void PeakObserver::observe(double time, const std::vector<Species*>& species) {
        for (const auto* s : species) {
            if (s->name() == species_name_ && s->count() > peak_value_) {
                peak_value_ = s->count();
                peak_time_ = time;
            }
        }
    }

    std::pair<double, size_t> PeakObserver::getPeak() const {
        return {peak_time_, peak_value_};
    }

    // Note: FunctionObserver<T> is implemented inline in the header to support templating.

} // namespace Stochastic
```

Listing 21: ./src/reaction.cpp

```cpp
//
// Created by Nicolai Bergulff on 12/06/2025.
//
#include "../include/Stochastic/stochastic.h"
#include <limits>
```

```cpp
#include <algorithm>
#include <iostream>

namespace Stochastic {

// === ReactionBuilder === //
ReactionBuilder::ReactionBuilder() : rate_(0.0), has_rate_(false) {}

ReactionBuilder::ReactionBuilder(size_t species_id)
    : ReactionBuilder() {
    reactant_ids_.emplace_back(species_id, 1);
}

ReactionBuilder::ReactionBuilder(const std::vector<size_t>& reactants)
    : rate_(0.0), has_rate_(false) {
    for (size_t id : reactants)
        reactant_ids_.emplace_back(id, 1);
}

ReactionBuilder ReactionBuilder::operator+(const Species& species) const {
    ReactionBuilder result(*this);
    result.reactant_ids_.emplace_back(species.id(), 1);
    return result;
}

ReactionBuilder ReactionBuilder::operator>>(double rate) const {
    ReactionBuilder result(*this);
    result.rate_ = rate;
    result.has_rate_ = true;
    return result;
}

ReactionBuilder ReactionBuilder::operator>>=(const Species& product) const {
    ReactionBuilder result(*this);
    result.product_ids_.emplace_back(product.id(), 1);
    return result;
}

ReactionBuilder ReactionBuilder::operator>>=(const ReactionBuilder& products) const {
    ReactionBuilder result(*this);
    result.product_ids_.insert(result.product_ids_.end(),
                               products.reactant_ids_.begin(),
                               products.reactant_ids_.end());
    return result;
}

const std::vector<std::pair<size_t, size_t>>& ReactionBuilder::reactants() const { return reactant_ids_; }
const std::vector<std::pair<size_t, size_t>>& ReactionBuilder::products() const { return product_ids_; }
double ReactionBuilder::rate() const { return rate_; }
bool ReactionBuilder::has_rate() const { return has_rate_; }

// === Reaction === //
Reaction::Reaction(const std::vector<std::pair<size_t, size_t>>& reactants,
                   const std::vector<std::pair<size_t, size_t>>& products,
                   double rate, size_t id)
    : reactant_ids_(reactants), product_ids_(products),
      rate_(rate), current_delay_(std::numeric_limits<double>::infinity()), id_(id) {}

void Reaction::set_delay(double delay) { current_delay_ = delay; }
```

```
65
66  const std::vector<std::pair<size_t, size_t>>& Reaction::reactants() const { return
    ↪reactant_ids_; }
67  const std::vector<std::pair<size_t, size_t>>& Reaction::products() const { return product_ids_; }
68  double Reaction::rate() const { return rate_; }
69  double Reaction::delay() const { return current_delay_; }
70  size_t Reaction::id() const { return id_; }
71
72  bool Reaction::can_proceed(const std::vector<std::shared_ptr<Species>>& species) const {
73      for (const auto& [id, count] : reactant_ids_)
74          if (species[id]->count() < count)
75              return false;
76      return true;
77  }
78
79  double Reaction::calculate_propensity(const std::vector<std::shared_ptr<Species>>& species)
    ↪const {
80      double propensity = rate_;
81      for (const auto& [id, count] : reactant_ids_) {
82          size_t available = species[id]->count();
83          if (available < count)
84              return 0.0;
85
86          double term = 1.0;
87          for (size_t i = 0; i < count; ++i)
88              term *= (available - i);
89
90          propensity *= term;
91      }
92
93      std::cout << "[DEBUG] Reaction " << id_ << " rate=" << rate_ << ", reactants:";
94      for (const auto& [id, count] : reactant_ids_) {
95          std::cout << " " << species[id]->name() << "=" << species[id]->count()
96                    << " (need " << count << ")";
97      }
98      std::cout << "     propensity = " << propensity << "\n";
99
100     return propensity;
101 }
102
103 void Reaction::execute(std::vector<std::shared_ptr<Species>>& species) const {
104     if (!can_proceed(species))
105         throw SimulationException("Cannot execute reaction: insufficient reactants");
106
107     for (const auto& [id, count] : reactant_ids_)
108         for (size_t i = 0; i < count; ++i)
109             species[id]->decrement_count();
110
111     for (const auto& [id, count] : product_ids_)
112         for (size_t i = 0; i < count; ++i)
113             species[id]->increment_count();
114 }
115
116 bool ReactionComparator::operator()(const std::shared_ptr<Reaction>& a, const
    ↪std::shared_ptr<Reaction>& b) const {
117     return a->delay() > b->delay();
118 }
119
120 } // namespace Stochastic
```

Listing 22: ./CMakeLists.txt

```
cmake_minimum_required(VERSION 3.31)
project(ExamProject)

set(CMAKE_CXX_STANDARD 23)

# Include all headers
include_directories(include)

# Define the core simulation library (no main() files here!)
add_library(ExamProject STATIC
        src/species.cpp
        src/reaction.cpp
        src/vessel.cpp
        src/simulator.cpp
        src/visualization.cpp
        src/observer.cpp

        examples/covid_seihr.cpp         #     library only
        examples/circadian_rythm.cpp     #     library only
)

# Link public headers for CLion indexing
target_include_directories(ExamProject PUBLIC include)

# === Examples with main() ===
add_executable(seir examples/seihr_main.cpp)
target_link_libraries(seir ExamProject)

add_executable(circadian examples/circadian_main.cpp)
target_link_libraries(circadian ExamProject)

add_executable(main examples/main.cpp)
target_link_libraries(main ExamProject)


# === Benchmark ===
add_executable(benchmark benchmarks/benchmark.cpp)
target_link_libraries(benchmark ExamProject)
```